

ReconForge

INHALTSVERZEICHNIS

1. Einleitung
 - 1.1 Projektbeschreibung
 - 1.2 Projektziel
 - 1.3 Verwendete Technologien
2. Requirements Engineering
 - 2.1 Brainstorming und Sammlung der Anforderungen
 - 2.2 Anforderungen
 - 2.2.1 Funktionale Anforderungen
 - 2.2.2 Nicht-funktionale Anforderungen
 - 2.3 Umsetzung in Notion und Jira
3. UML-Diagramme
 - 3.1 Verwendetes Werkzeug
 - 3.2 Übersicht der Diagramme
 - 3.3 Beschreibung der Diagramme
 - 3.3.1 Use-Case-Diagramm
 - 3.3.2 Class-Diagramm
 - 3.3.3 Activity-Diagramm
 - 3.3.4 Sequence-Diagramm
 - 3.3.5 Component-Diagramm
 - 3.4 Reflexion
4. Vorgehensmodell
 - 4.1 Vergleich von Scrum und Kanban
 - 4.2 Entscheidung für Kanban
 - 4.3 Kanban Board und Tasks
 - 4.4 Reflexion

5. AI / Vibe Coding

5.1 Ziel des Vibe Coding

5.2 Vergleich der verwendeten AI-Werkzeuge

5.3 Entscheidung für Codex

5.4 Vorgehensweise bei der Prompt-Erstellung

5.5 Beschreibung der verwendeten Prompts

5.5.1 Prompt 1 – Projektstruktur und Grundarchitektur

5.5.2 Prompt 2 – CLI-Grundlage und Logging

5.5.3 Prompt 3 – Domain Scan

5.5.4 Prompt 4 – Subdomain Discovery

5.5.5 Prompt 5 – IP Resolution

5.5.6 Prompt 6 – Safe Port Scan

5.5.7 Prompt 7 – Export-System mit Factory Pattern

5.5.8 Prompt 8 – Unit Testing Support

5.5.9 Prompt 9 – CLI-Banner und visuelles Design

5.5.10 Prompt 10 – Terminalausgabe als Standard und optionaler Export

5.6 Implementierung mit Codex

5.7 Ergebnisse der Implementierung

5.8 Reflexion zum Vibe Coding

6. Fazit

6.1 Zusammenfassung

6.2 Erfahrungen

6.3 Ausblick

1. Einleitung

1.1 Projektbeschreibung

Im Rahmen des Moduls Softwaretechnik wurde das Projekt **ReconForge** ausgewählt. ReconForge ist ein modulares Reconnaissance-Tool, das für Lern-

und Demonstrationszwecke im Bereich Softwaretechnik und IT-Sicherheit entwickelt wird.

Das Tool soll grundlegende Reconnaissance-Funktionen bereitstellen, zum Beispiel das Verarbeiten einer Domain, das Finden von Subdomains, die Auflösung von Domains in IP-Adressen, einfache Port-Scans sowie den Export der Ergebnisse in verschiedenen Formaten, zum Beispiel JSON, CSV, XML, HTML und YAML.

Der Schwerpunkt des Projekts liegt nicht nur auf der technischen Umsetzung, sondern vor allem auf der Anwendung verschiedener Methoden der Softwaretechnik. Dazu gehören Requirements Engineering, UML-Diagramme, ein geeignetes Vorgehensmodell, Testing, Architekturüberlegungen sowie der reflektierte Einsatz von KI-Werkzeugen im Rahmen von Vibe Coding.

1.2 Projektziel

Ziel des Projekts ist es, ein kleines, aber strukturiertes Softwareprojekt zu planen, zu dokumentieren und schrittweise umzusetzen. ReconForge soll dabei als Beispielprojekt dienen, um typische Schritte eines Softwareentwicklungsprozesses praktisch anzuwenden.

Das Projekt verfolgt folgende Ziele:

- Planung und Dokumentation der Anforderungen
- Erstellung einer modularen Projektstruktur
- Umsetzung zentraler Reconnaissance-Funktionen
- Verwendung eines Kanban Boards zur Aufgabenverwaltung
- Dokumentation der Entwicklungsschritte
- Anwendung von Tests und Qualitätssicherung
- Nachvollziehbare Ablage des Projekts in einem GitHub-Repository

1.3 Verwendete Technologien

Für die Umsetzung von **ReconForge** wird **C# mit .NET** verwendet. .NET ermöglicht die Entwicklung einer plattformübergreifenden Anwendung, die grundsätzlich auf **Windows, Linux und macOS** ausgeführt werden kann.

Da ReconForge als **Command-Line-Tool** entwickelt wird, wird **Spectre.Console.Cli** eingesetzt. Diese Bibliothek unterstützt die strukturierte

Verarbeitung von Befehlen, Argumenten und Optionen. Dadurch kann die CLI übersichtlich aufgebaut werden, zum Beispiel durch Optionen wie `--scan`, `--verbose`, `--export`, `--format` und `--output`.

Für die Versionsverwaltung wird **Git** in Verbindung mit **GitHub** verwendet. Das GitHub-Repository dient außerdem dazu, den Entwicklungsstand, die Commits und die Projektstruktur nachvollziehbar zu dokumentieren.

Die Projektplanung und Aufgabenverwaltung erfolgt mit **Jira** in Form eines Kanban Boards. Dort werden die aus den Anforderungen abgeleiteten Aufgaben organisiert und nach ihrem Bearbeitungsstatus verwaltet.

Für die schriftliche Projektdokumentation und die Strukturierung der Anforderungen wird **Notion** verwendet. Dort werden Inhalte wie Projektbeschreibung, Anforderungen, Notizen und Zwischenergebnisse gesammelt und übersichtlich dokumentiert.

Für das Brainstorming und die erste Ideensammlung wird **Miro** als digitales Whiteboard eingesetzt. Dort werden funktionale und nicht-funktionale Anforderungen sowie Projektziele visuell gesammelt und strukturiert.

Die UML-Diagramme werden mit **draw.io** erstellt. Für das Logging innerhalb der Anwendung wird **Microsoft.Extensions.Logging** eingesetzt. Für den Export werden eigene Exporter für JSON, CSV, XML, HTML und YAML verwendet. Der **JSON-Export** nutzt **System.Text.Json**. Für Unit Tests wird **xUnit** verwendet.

Zusätzlich werden KI-Werkzeuge unterstützend eingesetzt. **ChatGPT** wird zur Formulierung von Dokumentationstexten und zur Korrektur sprachlicher Fehler verwendet. **Codex** wird im Rahmen von Vibe Coding und zur AI-gestützten Umsetzung einzelner Programmieraufgaben genutzt. Die erzeugten Texte und Codevorschläge werden geprüft, angepasst und in den Projektkontext eingeordnet.

Verwendete Technologien und Werkzeuge

Technologie / Werkzeug	Verwendung im Projekt
C# / .NET	Programmiersprache und Laufzeitumgebung für die Umsetzung von ReconForge
Spectre.Console.Cli	Aufbau des Command-Line-Interfaces mit Befehlen, Argumenten und Optionen
Git	Versionsverwaltung des Projekts

Technologie / Werkzeug	Verwendung im Projekt
GitHub	Ablage des Repositorys und Nachvollziehbarkeit der Entwicklung
Jira	Kanban Board und Aufgabenverwaltung
Notion	Dokumentation und Strukturierung der Anforderungen
draw.io	Erstellung der UML-Diagramme
Microsoft.Extensions.Logging	Logging innerhalb der Anwendung
System.Text.Json	Serialisierung der Scan-Ergebnisse im JSON-Format
Eigene Exporter	Export der Scan-Ergebnisse in JSON, CSV, XML, HTML und YAML
xUnit	Erstellung und Ausführung von Unit Tests
ChatGPT	Unterstützung bei der Formulierung von Dokumentationstexten und bei der Korrektur sprachlicher Fehler
Codex	Unterstützung beim Vibe Coding und bei der AI-gestützten Umsetzung einzelner Programmieraufgaben
Miro	Brainstorming, Ideensammlung und visuelle Strukturierung der ersten Anforderungen

Tabelle 1: Verwendete Technologien und Werkzeuge

2. Requirements Engineering

2.1 Brainstorming und Sammlung der Anforderungen

Zu Beginn der Requirements-Engineering-Phase wurde ein Brainstorming mit **Miro** durchgeführt, um erste Ideen und mögliche Anforderungen für das Projekt **ReconForge** zu sammeln. Miro wurde dabei als digitales Whiteboard verwendet, um Funktionen, technische Eigenschaften und erste Zusammenhänge visuell zu strukturieren.

Dabei wurden zunächst alle wichtigen Ideen notiert, ohne sie direkt zu bewerten oder auszusortieren. Im Fokus standen Fragen wie: Welche Funktionen soll ein einfaches Reconnaissance-Tool anbieten? Welche Eingaben soll der Benutzer machen können? Welche Ergebnisse sollen erzeugt werden?

Außerdem wurde überlegt, welche technischen Eigenschaften für das Projekt wichtig sind, zum Beispiel eine modulare Struktur, Logging, Testbarkeit und eine einfache Bedienung über die Kommandozeile.

Nach dem Brainstorming wurden die gesammelten Ideen geordnet und in **funktionale Anforderungen** und **nicht-funktionale Anforderungen** unterteilt. Die funktionalen Anforderungen beschreiben die konkreten Funktionen des Systems, während die nicht-funktionalen Anforderungen technische und qualitative Eigenschaften des Projekts beschreiben.

Ergebnisse des Brainstormings

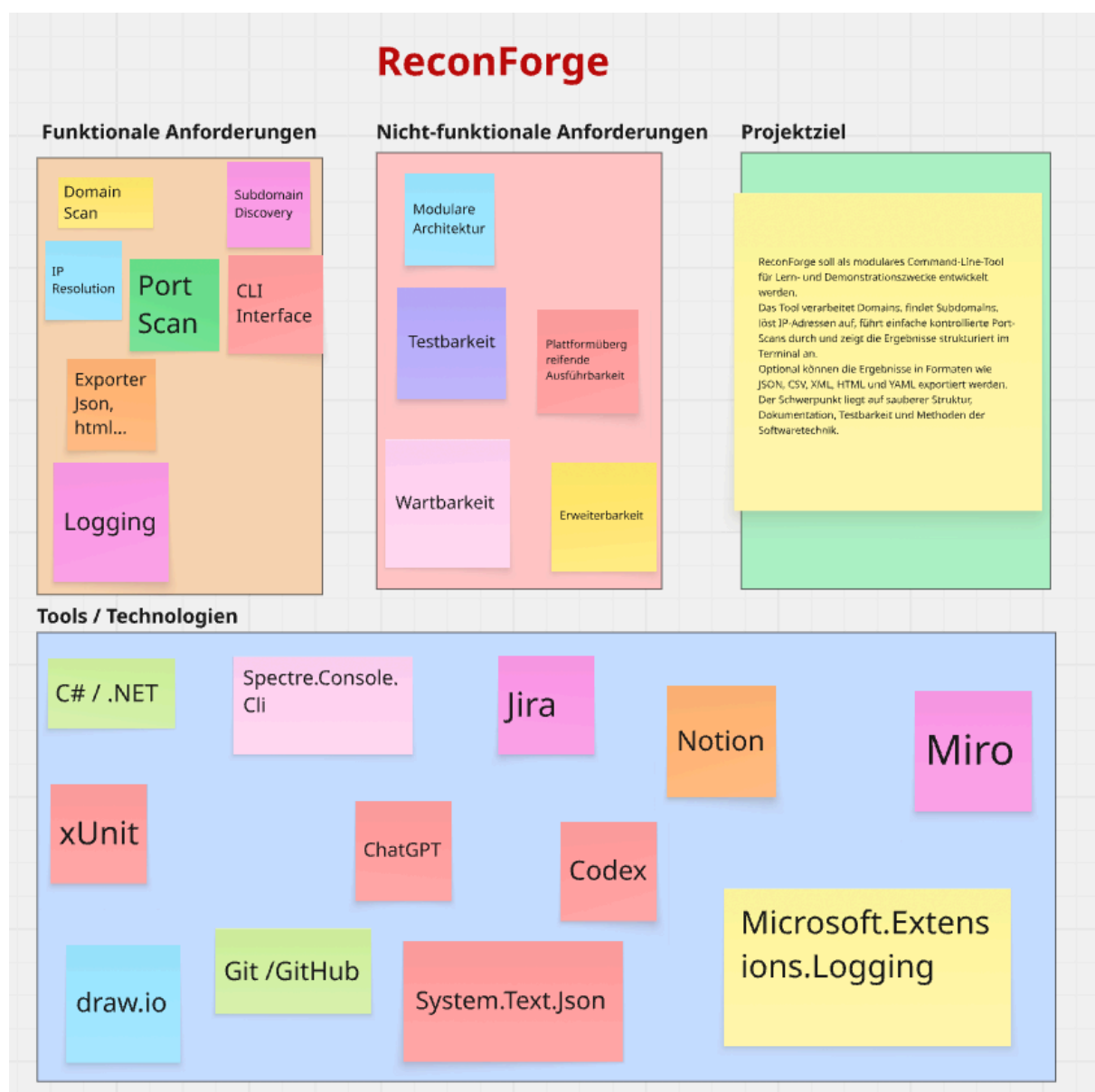


Abbildung 1: Brainstorming und erste Anforderungssammlung in Miro

2.2 Anforderungen

2.2.1 Funktionale Anforderungen

Funktionale Anforderungen beschreiben, welche konkreten Funktionen das System bereitstellen soll. Bei ReconForge stehen dabei die grundlegenden Reconnaissance-Funktionen und die Bedienung über die Kommandozeile im Mittelpunkt.

ID	Anforderung	Beschreibung	Priorität
REQ-001	Domain Scan	Das System soll eine Domain als Eingabe verarbeiten können.	Hoch
REQ-002	Subdomain Discovery	Das System soll mögliche Subdomains zu einer angegebenen Domain finden oder verarbeiten können.	Hoch
REQ-003	IP Resolution	Das System soll Domains und Subdomains in IP-Adressen auflösen können.	Hoch
REQ-004	Port Scan	Das System soll ausgewählte Ports einer IP-Adresse prüfen können.	Hoch
REQ-005	Export der Scan-Ergebnisse	Das System soll die Scan-Ergebnisse strukturiert exportieren können. JSON ist dabei das zentrale Mindestformat.	Mittel
REQ-006	CLI Interface	Das System soll über die Kommandozeile bedienbar sein.	Hoch
REQ-007	Log-Ausgabe / Logging	Das System soll wichtige Abläufe, Fehler und Ergebnisse während der Ausführung als Log-Ausgabe bereitstellen.	Mittel

Tabelle 2: Funktionale Anforderungen von ReconForge

Begründung der funktionalen Anforderungen

Die Anforderungen Domain Scan, Subdomain Discovery, IP Resolution und Port Scan bilden die Kernfunktionen von ReconForge. Sie beschreiben direkt, welche Reconnaissance-Aufgaben das Tool ausführen soll.

Der Export der Scan-Ergebnisse wird als funktionale Anforderung eingeordnet, weil das System die erzeugten Ergebnisse in einem strukturierten Format speichern können soll. JSON ist dabei das zentrale Mindestformat. Die zusätzlich umgesetzten Formate CSV, XML, HTML und YAML erweitern diese Funktion und unterstützen die Erweiterbarkeit des Projekts.

Das CLI Interface ist ebenfalls funktional, da es beschreibt, wie der Benutzer mit dem Tool interagiert.

Logging wurde hier als funktionale Anforderung aufgenommen, weil ReconForge während der Ausführung konkrete Log-Ausgaben bereitstellen soll. Diese Ausgaben sind ein sichtbarer Teil des Systemverhaltens und helfen dabei, Scan-Abläufe, Fehler und Ergebnisse nachzuvollziehen.

2.2.2 Nicht-funktionale Anforderungen

Nicht-funktionale Anforderungen beschreiben keine einzelnen Benutzerfunktionen, sondern technische und qualitative Eigenschaften des Systems. Sie legen fest, wie das System aufgebaut sein soll und welche Qualitätsmerkmale bei der Umsetzung wichtig sind.

ID	Anforderung	Beschreibung	Priorität
NFR-001	Modulare Architektur	Das Projekt soll modular aufgebaut sein, damit einzelne Funktionen getrennt entwickelt, getestet und erweitert werden können.	Hoch
NFR-002	Testbarkeit	Zentrale Funktionen sollen so aufgebaut sein, dass sie durch Unit Tests überprüft werden können.	Mittel
NFR-003	Plattformübergreifende Ausführbarkeit	Die Anwendung soll grundsätzlich auf Windows, Linux und macOS lauffähig sein.	Mittel
NFR-004	Wartbarkeit	Der Code soll übersichtlich strukturiert sein, damit Änderungen und Fehlerbehebungen einfacher möglich sind.	Hoch

ID	Anforderung	Beschreibung	Priorität
NFR-005	Erweiterbarkeit	Neue Scan-Funktionen oder Exportformate sollen später einfacher ergänzt werden können.	Mittel
NFR-006	Nachvollziehbarkeit	Die Abläufe des Systems sollen verständlich und nachvollziehbar sein, insbesondere bei Fehlern oder Scan-Ergebnissen.	Mittel

Tabelle 3: Nicht-funktionale Anforderungen von ReconForge

Begründung der nicht-funktionalen Anforderungen

Die modulare Architektur ist eine nicht-funktionale Anforderung, weil sie nicht eine einzelne Funktion beschreibt, sondern die Struktur des gesamten Projekts betrifft. Durch die modulare Aufteilung können einzelne Bestandteile wie Domain Scan, IP Resolution, Port Scan und das Export-System sauber getrennt entwickelt werden.

Testbarkeit wurde als nicht-funktionale Anforderung eingeordnet, weil Unit Tests keine direkte Benutzerfunktion des Tools sind. Sie dienen der Qualitätssicherung und helfen dabei, zentrale Funktionen zuverlässig zu überprüfen.

Die plattformübergreifende Ausführbarkeit ist ebenfalls nicht-funktional, da sie beschreibt, auf welchen Betriebssystemen die Anwendung grundsätzlich laufen soll. Wartbarkeit und Erweiterbarkeit betreffen die langfristige Qualität des Codes und erleichtern spätere Änderungen oder Erweiterungen.

Nachvollziehbarkeit beschreibt die Qualität, dass Abläufe und Fehler verständlich bleiben. Diese Eigenschaft wird unter anderem durch Logging unterstützt, ist aber selbst ein übergeordnetes Qualitätsmerkmal des Systems.

2.3 Umsetzung in Notion und Jira

Nach dem Brainstorming und der Einteilung der Anforderungen in funktionale und nicht-funktionale Anforderungen wurden die Ergebnisse in zwei verschiedenen Werkzeugen abgebildet: **Notion** und **Jira**. Ziel war es, die Anforderungen nicht nur textlich zu beschreiben, sondern sie auch praktisch in geeigneten Werkzeugen zu strukturieren und für die weitere Umsetzung vorzubereiten.

Zuerst wurden die Anforderungen in **Notion** dokumentiert. Notion wurde dabei als flexibles Dokumentationswerkzeug verwendet. Die Anforderungen wurden in einer Tabelle gesammelt und mit mehreren Beschreibungsattributen ergänzt. Dazu gehören zum Beispiel **ID, Kurzbeschreibung, Beschreibung, Priorität, Status, Kategorie, Datum, Autor, Abnahmekriterium, Abhängigkeit** und **Historie**.

Durch diese Attribute konnten die Anforderungen genauer beschrieben und nachvollziehbar dokumentiert werden. Die Priorität zeigt, wie wichtig eine Anforderung für das Projekt ist. Der Status zeigt den aktuellen Bearbeitungsstand. Das Abnahmekriterium beschreibt, wann eine Anforderung als erfüllt angesehen werden kann. Die Abhängigkeit macht sichtbar, ob eine Anforderung von einem anderen Modul oder einer anderen Funktion abhängig ist.

Anschließend wurden die Anforderungen in **Jira** weiterverarbeitet. Jira wurde als professionelles Werkzeug für die Aufgabenverwaltung eingesetzt. Ursprünglich wurde dafür ein **Scrum Board** vorbereitet. Dabei wurden die Anforderungen zunächst in Sprints eingeordnet. Nach weiterer Analyse wurde jedoch entschieden, das Vorgehensmodell auf **Kanban** umzustellen.

Der Grund dafür ist, dass Scrum mit festen Zeitabschnitten arbeitet, sogenannten Sprints. Für jeden Sprint müsste vorher ein bestimmter Zeitraum festgelegt werden, zum Beispiel eine oder zwei Wochen. Da ReconForge jedoch schrittweise und teilweise mit Unterstützung von **Codex im Rahmen von Vibe Coding** umgesetzt wird, ist es schwierig, den Aufwand einzelner Aufgaben im Voraus genau einzuschätzen.

Kanban eignet sich für dieses Projekt besser, weil die Aufgaben flexibel nacheinander bearbeitet werden können. Es gibt keine festen Sprint-Zeiträume und keine Verpflichtung, bestimmte Aufgaben innerhalb eines vorher definierten Zeitfensters abzuschließen. Stattdessen wird der Arbeitsfluss über ein Kanban Board sichtbar gemacht. Die Aufgaben können je nach Fortschritt von **Backlog** über **To Do** und **In Progress** bis nach **Review** und **Done** verschoben werden.

Die Anforderungen wurden in Jira in konkrete Aufgaben für das Kanban Board überführt. Dadurch wurde aus einer fachlichen oder technischen Anforderung eine umsetzbare Entwicklungsaufgabe. Ein Beispiel dafür ist die Anforderung **Domain Scan**. In Notion wird diese Anforderung beschrieben und mit Attributen dokumentiert. In Jira wird daraus eine konkrete Aufgabe wie **Domain Scan implementieren**.

Durch die Kombination von Notion und Jira konnte der Unterschied zwischen Requirements-Dokumentation und Aufgabenverwaltung praktisch nachvollzogen werden. Notion diente vor allem der strukturierten Beschreibung der Anforderungen, während Jira für die Planung, Priorisierung und Nachverfolgung der konkreten Umsetzung genutzt wurde. Die Umstellung von Scrum auf Kanban zeigt außerdem, dass das Vorgehensmodell an die tatsächliche Arbeitsweise des Projekts angepasst wurde.

Req ID	Kurzbeschreibung	Beschreibung	Priorität	Status	Kategorie	Datum	Author	Abnahmekriterium	Abhängigkeit	Historie
REQ-001	Domain Scan	The system must analyze target domains and collect DNS-related information required for reconnaissance workflows.	Hoch	Implementiert	Kernfunktion	May 16, 2026	Mohamed Ali	Valid domain scan result	DNS Module	Initial Version
REQ-002	Subdomain Discovery	The system must discover and store subdomains associated with a target domain during reconnaissance operations.	Hoch	In Entwicklung	Network	May 16, 2026	Mohamed Ali	Subdomains are correctly detected	DNS Resolver	Initial Version
REQ-003	IP Resolution	The system must resolve domains and subdomains to their corresponding IP addresses.	Hoch	Geplant	Network	May 16, 2026	Mohamed Ali	Correct IP address resolution	Socket Module	Initial Version
REQ-004	Port Scan	The system must scan common network ports and identify accessible services on target hosts.	Hoch	In Test	Network	May 16, 2026	Mohamed Ali	Open ports are detected correctly	Network Scanner	Initial Version
REQ-005	Export der Scan-Ergebnisse	The system must export reconnaissance results in structured formats such as JSON, CSV, XML, HTML, and YAML, for further processing and documentation.	Hoch	In Entwicklung	Export	May 16, 2026	Mohamed Ali	Scan results can be exported successfully in the selected format	Export Module	Initial Version
REQ-006	CLI Interface	The system must provide a command-line interface for executing scans and managing reconnaissance tasks.	Normal	Implementiert	CLI	May 16, 2026	Mohamed Ali	CLI commands execute successfully	Core Module	Initial Version
REQ-007	Logging System	The system must generate structured log files for scan activities and system events.	Normal	Geplant	Kernfunktion	May 16, 2026	Mohamed Ali	Log files are generated correctly	Logging Module	Initial Version
REQ-008	Modular Architecture	The system must use a modular architecture to support maintainability and future feature extensions.	Hoch	In Entwicklung	Kernfunktion	May 16, 2026	Mohamed Ali	Modules are separated correctly	Core Architecture	Initial Version
REQ-009	Unit Testing Support	The system must support automated unit testing to improve software quality and maintainability.	Normal	Geplant	Kernfunktion	May 16, 2026	Mohamed Ali	Unit tests execute successfully	Testing Framework	Initial Version

Abbildung 2: Dokumentation der Anforderungen in Notion

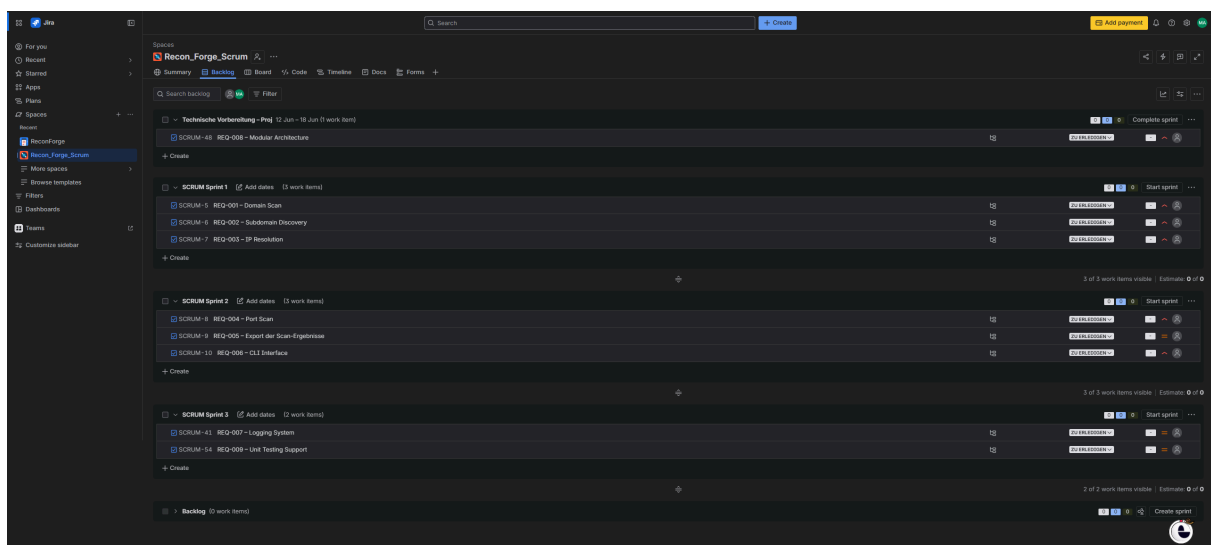


Abbildung 3: Jira Scrum Board mit Aufgaben

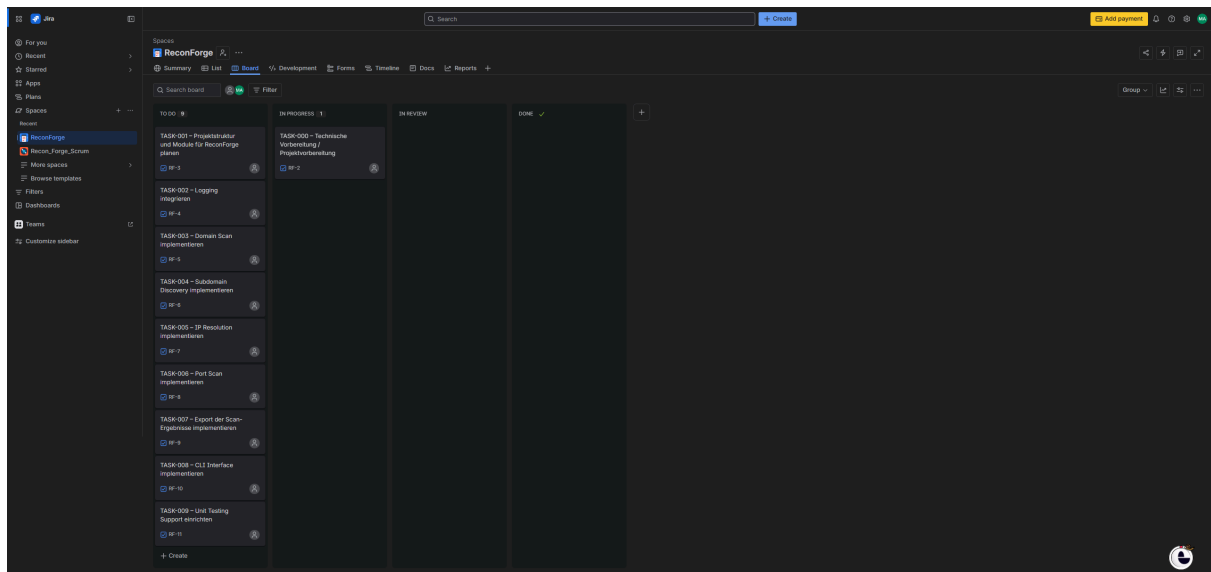


Abbildung 4: Jira Kanban Board mit Aufgaben

3. UML-Diagramme

3.1 Verwendetes Werkzeug

Für die Erstellung der UML-Diagramme wurde **draw.io** verwendet. Das Werkzeug eignet sich gut für die visuelle Modellierung verschiedener UML-Diagrammtypen und ermöglicht eine übersichtliche Darstellung von Funktionen, Abläufen, Klassen und Komponenten.

Die Diagramme wurden erstellt, um das Projekt **ReconForge** aus unterschiedlichen Perspektiven zu betrachten. Dadurch kann das System sowohl aus Sicht des Benutzers als auch aus technischer Sicht besser verstanden und geplant werden.

3.2 Übersicht der Diagramme

Für ReconForge wurden mehrere UML-Diagramme erstellt. Jedes Diagramm zeigt einen anderen Blick auf das System.

Diagramm	Zweck
Use-Case-Diagramm	Darstellung der wichtigsten Benutzeraktionen im System
Class-Diagramm	Darstellung zentraler Klassen, Attribute, Methoden und Beziehungen
Activity-Diagramm	Darstellung des Ablaufs eines typischen Scan-Prozesses

Diagramm	Zweck
Sequence-Diagramm	Darstellung der Kommunikation zwischen Benutzer, CLI und internen Modulen
Component-Diagramm	Darstellung der modularen technischen Struktur des Systems

Tabelle 4: Übersicht der UML-Diagramme

3.3 Beschreibung der Diagramme

3.3.1 Use-Case-Diagramm

Das Use-Case-Diagramm zeigt die wichtigsten Funktionen, die ein Benutzer mit ReconForge ausführen kann. Der Akteur ist ein **Security Researcher**, der das Tool über die Kommandozeile verwendet und verschiedene Reconnaissance-Aktionen startet.

Zu den wichtigsten Anwendungsfällen gehören **Domain Scan**, **Subdomain Discovery**, **IP Resolution**, **Port Scan** und **Export Scan Results**. Diese Anwendungsfälle entsprechen den zentralen funktionalen Anforderungen des Projekts.

Zusätzlich zeigt das Diagramm unterstützende Anwendungsfälle wie **Validate Domain Input**, **Load Configuration** und **Generate Scan Report**. Diese werden nicht immer direkt vom Benutzer gestartet, sind aber wichtige interne Schritte, die für die Ausführung bestimmter Funktionen benötigt werden.

Die Beziehungen mit `include` zeigen, dass bestimmte Funktionen immer einen zusätzlichen Schritt benötigen. Zum Beispiel muss bei mehreren Scan-Funktionen zuerst die Domain-Eingabe validiert werden. Beim Port Scan wird außerdem eine Scan-Konfiguration geladen. Beim Export der Ergebnisse wird ein Scan-Bericht erzeugt.

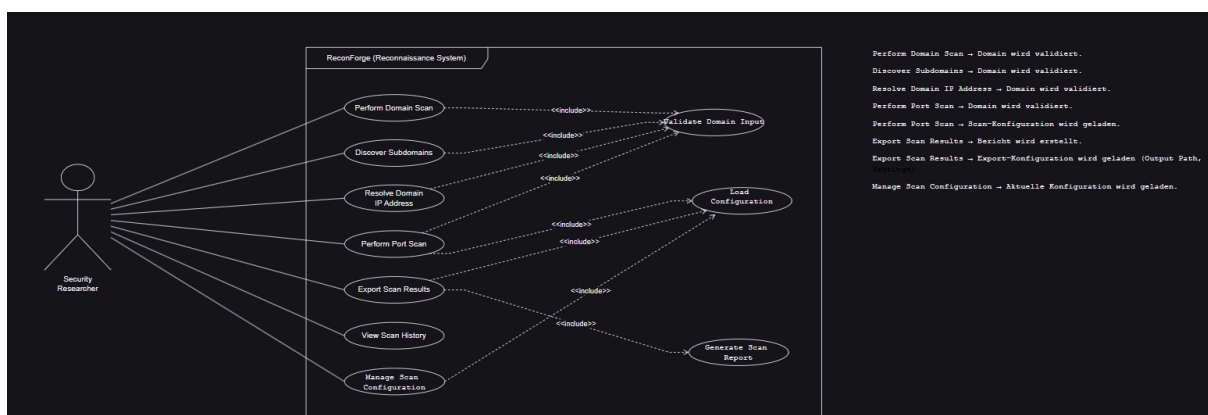


Abbildung 5: Use-Case-Diagramm von ReconForge

3.3.2 Class-Diagramm

Das Class-Diagramm beschreibt die wichtigsten Datenstrukturen und Beziehungen innerhalb von ReconForge. Es zeigt, welche Klassen im System vorgesehen sind und wie diese miteinander verbunden sind.

Zentrale Klassen sind **Domain**, **Subdomain**, **IpAddress**, **Port**, **DnsRecord**, **ScanConfiguration**, **ScanTask** und **ScanResult**.

Die Klasse **Domain** bildet den Ausgangspunkt des Systems. Eine Domain kann mehrere Subdomains, IP-Adressen und DNS-Einträge besitzen. Eine **Subdomain** kann wiederum einer IP-Adresse zugeordnet werden. Die Klasse **IpAddress** kann mehrere Ports enthalten, wodurch der Port Scan modelliert wird.

Die Klasse **DnsRecord** beschreibt DNS-Einträge einer Domain, zum Beispiel A-, AAAA-, MX- oder TXT-Einträge. Der Typ eines DNS-Eintrags wird über die Enumeration **DnsRecordType** dargestellt.

Die Klasse **ScanConfiguration** enthält Einstellungen für den Scan, zum Beispiel Timeout, Anzahl der Threads, maximale Ports und aktivierte Scan-Module. **ScanTask** beschreibt einen konkreten Scan-Vorgang, während **ScanResult** die Ergebnisse eines abgeschlossenen Scans enthält.

Das Class-Diagramm hilft dabei, die interne Struktur des Systems zu verstehen und zeigt, welche Objekte für die Umsetzung von ReconForge wichtig sind.

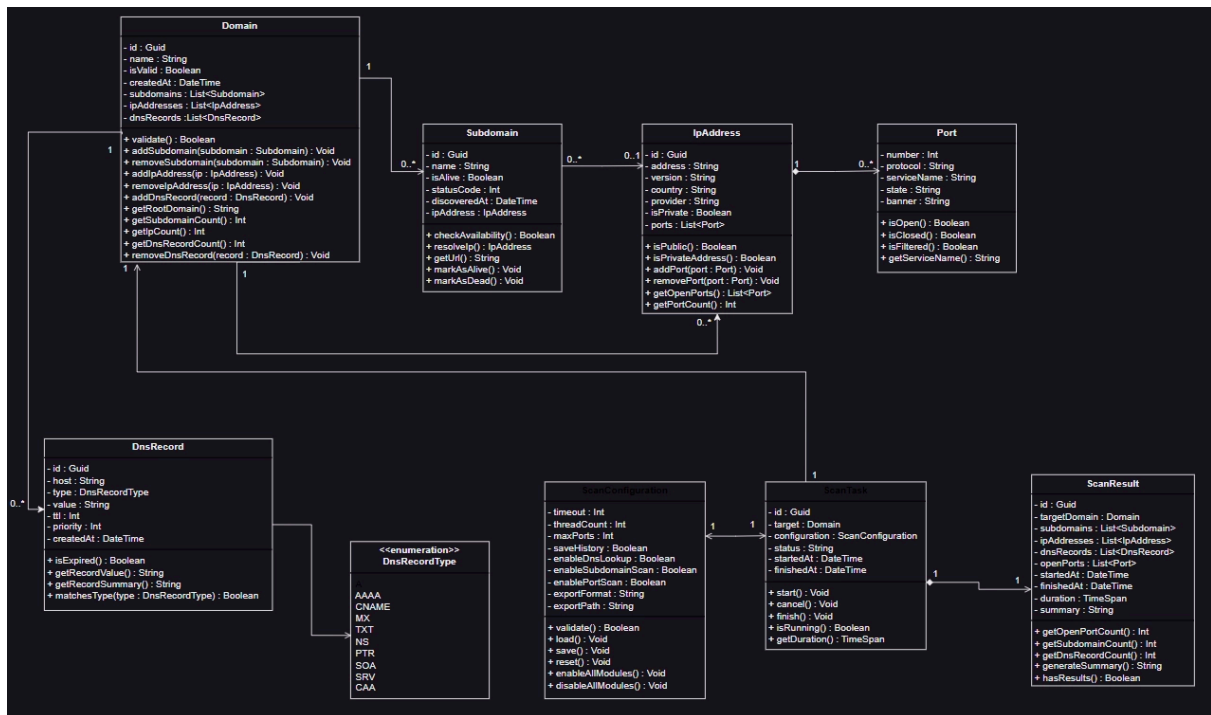


Abbildung 6: Class-Diagramm von ReconForge

3.3.3 Activity-Diagramm

Das Activity-Diagramm zeigt den Ablauf eines typischen Arbeitsprozesses in ReconForge. Der Prozess beginnt mit dem Start der CLI und der Auswertung der eingegebenen Optionen.

Danach wird abhängig von den eingegebenen Optionen entschieden, ob ein Scan gestartet, eine Konfiguration berücksichtigt oder ein Export vorbereitet werden soll. Dadurch zeigt das Diagramm nicht nur einen linearen Ablauf, sondern auch mögliche Entscheidungen innerhalb des Programms.

Beim Start eines Scans wird zuerst der eingegebene Domainname verarbeitet. Danach wird geprüft, ob die Domain gültig ist. Wenn die Domain ungültig ist, wird eine Fehlermeldung angezeigt und der Prozess endet. Wenn die Domain gültig ist, wird der Scan-Prozess fortgesetzt.

Anschließend werden die wichtigsten Scan-Schritte ausgeführt. Zuerst werden mögliche Subdomains ermittelt. Danach werden die Domain und die gefundenen Subdomains in IP-Adressen aufgelöst. Anschließend wird ein kontrollierter Port Scan mit einer kleinen vordefinierten Portliste durchgeführt. Die Ergebnisse werden danach zu einem strukturierten Scan-Ergebnis zusammengeführt.

Standardmäßig werden die Scan-Ergebnisse direkt im Terminal angezeigt. Dazu gehören Zielinformationen, gefundene Subdomains, aufgelöste IP-Adressen, offene Ports sowie eine Zusammenfassung des Scan-Ergebnisses.

Am Ende kann zusätzlich geprüft werden, ob der Benutzer einen Export angefordert hat. Wenn die Option `--export` verwendet wurde, wird das gewünschte Exportformat ausgewählt und die Ergebnisse werden in eine Datei exportiert. Wenn kein Export angefordert wurde, endet der Prozess nach der Terminalausgabe.

Das Activity-Diagramm zeigt damit den Ablauf des Programms inklusive Validierung, Scan-Schritten, Terminalausgabe, optionalem Export und möglichen Endpunkten.

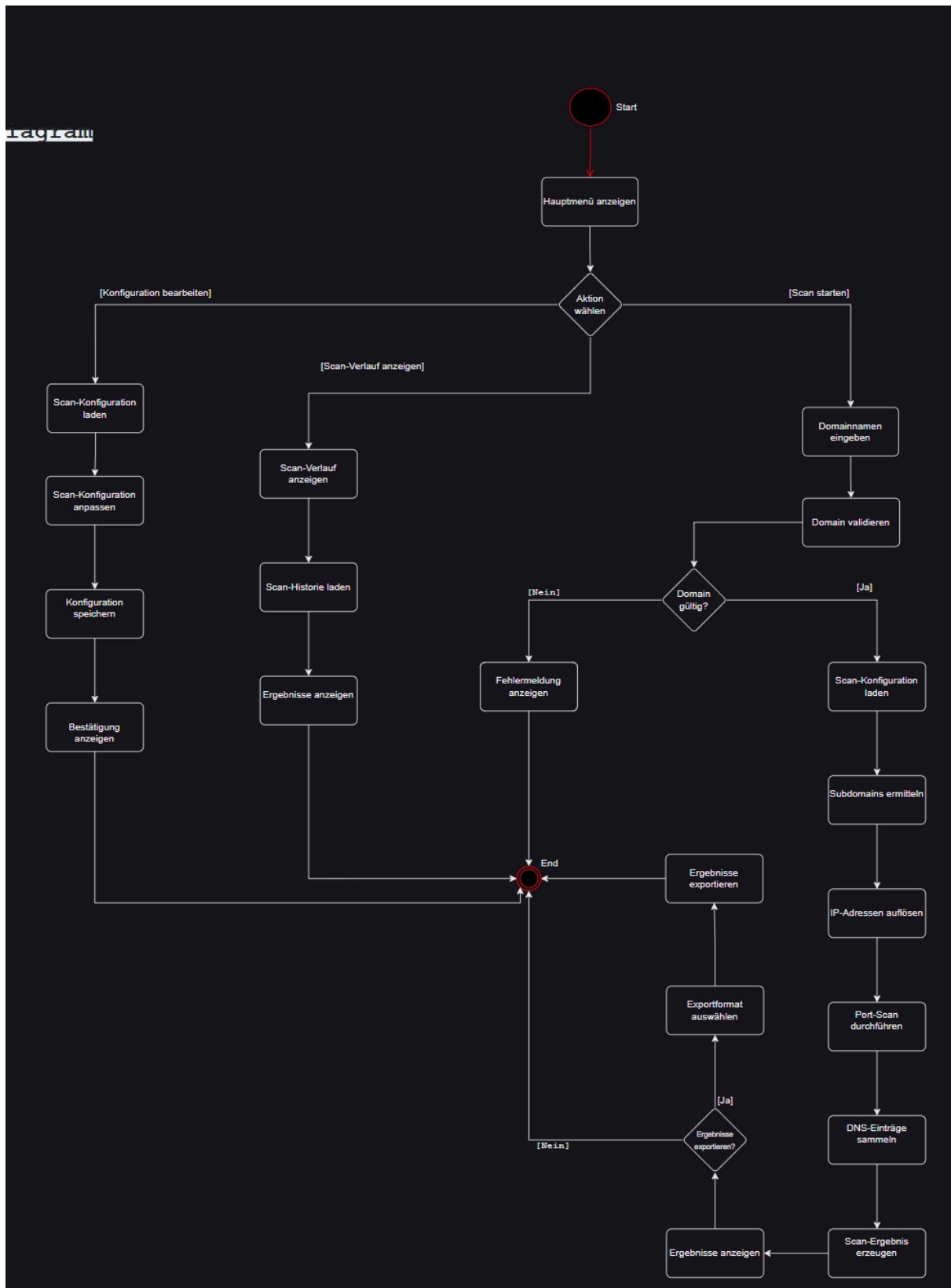


Abbildung 7: Activity-Diagramm des Scan-Prozesses

3.3.4 Sequence-Diagramm

Das Sequence-Diagramm stellt die zeitliche Reihenfolge der Kommunikation zwischen dem Benutzer, der CLI und den internen Modulen dar.

Der Prozess beginnt damit, dass der **Security Researcher** über die **CLI** einen Scan startet und einen Domainnamen eingibt. Danach wird die Domain durch den **DomainValidator** geprüft.

Das Diagramm enthält eine Alternative für zwei mögliche Fälle. Wenn die Domain ungültig ist, wird eine Fehlermeldung angezeigt. Wenn die Domain gültig ist, startet der eigentliche Scan-Prozess.

In diesem Fall ruft der **DomainScanner** nacheinander mehrere Komponenten auf. Der **SubdomainScanner** ermittelt Subdomains, der **IpResolver** löst IP-Adressen auf, der **PortScanner** prüft offene Ports und der **DnsCollector** sammelt DNS-Einträge.

Die Ergebnisse werden anschließend im **ScanResult** gespeichert. Danach werden die Ergebnisse standardmäßig im Terminal angezeigt. Optional können sie über den **ResultExporter** in ein ausgewähltes Dateiformat exportiert werden.

Das Sequence-Diagramm macht sichtbar, in welcher Reihenfolge die Komponenten miteinander kommunizieren und wie die einzelnen Schritte des Scan-Ablaufs zusammenhängen.



Abbildung 8: Sequence-Diagramm des Scan-Ablaufs

3.3.5 Component-Diagramm

Das Component-Diagramm zeigt die modulare technische Struktur von ReconForge. Das System ist in mehrere Komponenten aufgeteilt, die jeweils eine klare Verantwortung besitzen.

Die **CLI Component** bildet den Einstiegspunkt für den Benutzer. Sie verarbeitet Kommandozeilenoptionen, zeigt Meldungen und Ergebnisse im Terminal an und leitet Benutzeraktionen an die passenden Komponenten weiter.

Die **Scanning Component** ist für die Koordination der Scan-Abläufe zuständig. Sie verwendet unter anderem die **IP Component**, die **DNS Component** und die **Port Component**. Dadurch werden die einzelnen Aufgaben logisch voneinander getrennt.

Die **IP Component** enthält Funktionen zur IP-Auflösung und IP-Validierung. Die **DNS Component** ist für DNS-bezogene Aufgaben zuständig. Die **Port Component** enthält die Logik für den Port Scan und die Port-Validierung.

Die **Config Component** verwaltet Einstellungen des Scan-Vorgangs. Die **Result Component** sammelt und strukturiert die Scan-Ergebnisse. Die **Export Component** ist für den Export der Ergebnisse zuständig.

Im aktuellen Projekt ist JSON das zentrale Exportformat aus den Anforderungen. Die Export-Komponente wurde jedoch generisch umgesetzt, sodass zusätzlich CSV, XML, HTML und YAML unterstützt werden können. Dadurch können später weitere Exportformate ergänzt werden, ohne die Grundstruktur des Systems verändern zu müssen.

Diese Modellierung unterstützt die nicht-funktionale Anforderung **Erweiterbarkeit**. Neue Exporter können als eigene Klassen oder Module ergänzt werden, während die restlichen Komponenten weiterhin nur die allgemeine Export-Schnittstelle verwenden.

Das Component-Diagramm zeigt damit, dass ReconForge modular aufgebaut ist. Diese Aufteilung verbessert die Wartbarkeit, Testbarkeit und Erweiterbarkeit des Projekts.

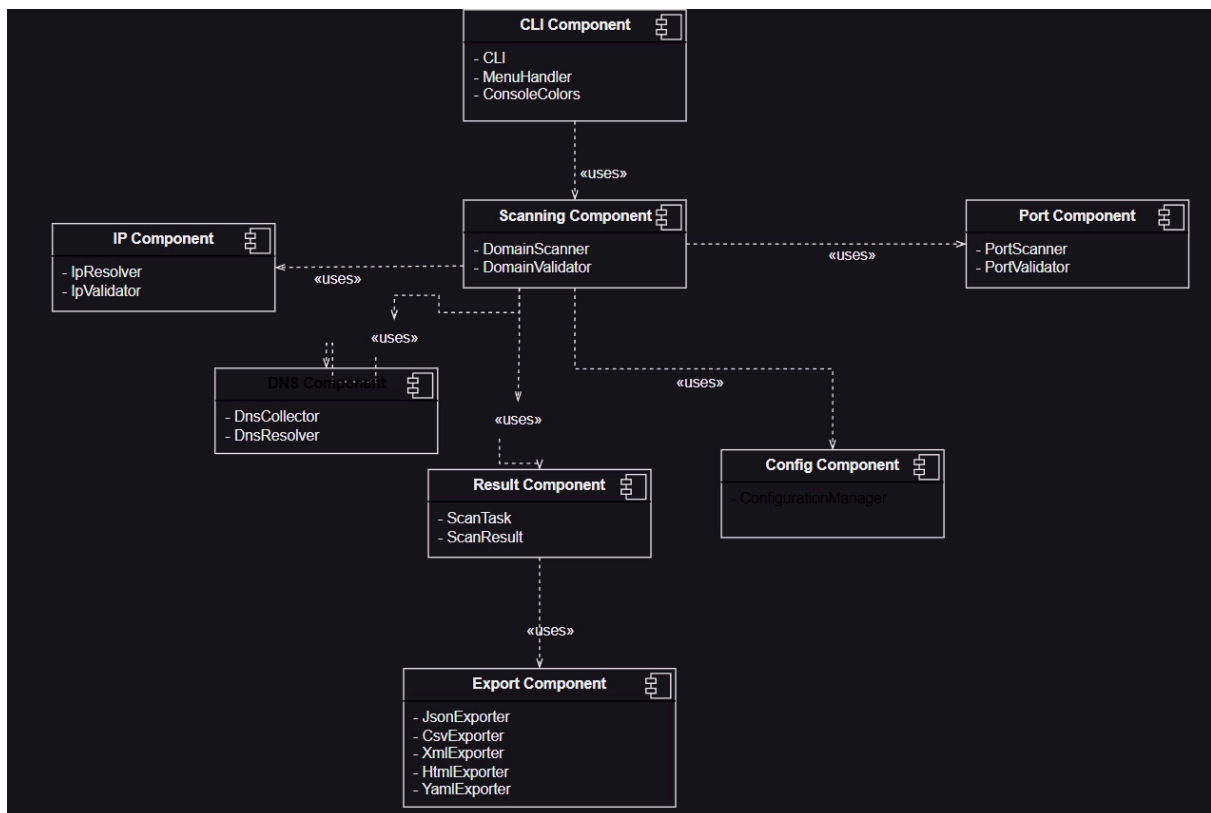


Abbildung 9: Component-Diagramm der modularen Architektur

3.4 Reflexion

Die UML-Diagramme haben geholfen, ReconForge vor der Implementierung besser zu planen und zu strukturieren. Durch die verschiedenen Diagrammtypen konnte das Projekt aus mehreren Perspektiven betrachtet werden.

Das Use-Case-Diagramm zeigt die Sicht des Benutzers und macht deutlich, welche Funktionen ReconForge anbieten soll. Das Class-Diagramm beschreibt die wichtigsten Datenstrukturen und Beziehungen im System. Das Activity-Diagramm zeigt den Ablauf eines typischen Scan-Prozesses. Das Sequence-Diagramm zeigt die Kommunikation zwischen den Komponenten während eines Scans. Das Component-Diagramm stellt die technische Aufteilung des Systems dar.

Besonders wichtig war das Component-Diagramm, weil es die modulare Architektur von ReconForge sichtbar macht. Dadurch wurde klarer, welche Bestandteile getrennt entwickelt und später erweitert werden können.

Insgesamt unterstützen die UML-Diagramme die Planung, Dokumentation und spätere Umsetzung des Projekts. Sie zeigen, dass ReconForge nicht nur als

einzelnes Programm, sondern als strukturiertes und modular aufgebautes Softwareprojekt betrachtet wird.

4. Vorgehensmodell

4.1 Vergleich von Scrum und Kanban

Für die Planung und Organisation des Projekts ReconForge wurden agile Vorgehensmodelle betrachtet. Besonders relevant waren dabei Scrum und Kanban.

Scrum strukturiert die Arbeit in feste Zeitabschnitte, sogenannte Sprints. Innerhalb eines Sprints wird eine bestimmte Menge an Aufgaben geplant und anschließend umgesetzt. Scrum arbeitet außerdem mit festen Rollen und regelmäßigen Meetings.

Kanban dagegen visualisiert den Arbeitsablauf über ein Board. Die Aufgaben werden in Spalten wie Backlog, To Do, In Progress, Review und Done eingeordnet. Im Gegensatz zu Scrum gibt es bei Kanban keine festen Sprint-Zeiträume. Dadurch können Aufgaben flexibel nacheinander bearbeitet werden.

Ein weiterer wichtiger Punkt bei Kanban ist der kontinuierliche Arbeitsfluss. Durch sogenannte WIP-Limits kann begrenzt werden, wie viele Aufgaben gleichzeitig bearbeitet werden. Dadurch bleibt der Arbeitsprozess übersichtlich und Engpässe können leichter erkannt werden.

4.2 Entscheidung für Kanban

Ursprünglich wurde überlegt, das Projekt mit Scrum zu organisieren. Dafür wurde zunächst ein Scrum Board vorbereitet und die Anforderungen wurden in einzelne Sprints eingeordnet.

Nach weiterer Analyse wurde jedoch entschieden, auf Kanban umzusteigen. Der Grund dafür ist, dass Scrum mit festen Sprint-Zeiträumen arbeitet. Für jeden Sprint müsste vorher ein bestimmter Zeitraum festgelegt werden, zum Beispiel eine oder zwei Wochen.

Da ReconForge schrittweise und teilweise mit Unterstützung von Codex im Rahmen von Vibe Coding umgesetzt wird, ist es schwierig, den Aufwand einzelner Aufgaben im Voraus genau einzuschätzen. Bei AI-gestützter Entwicklung können während der Umsetzung neue Erkenntnisse, Probleme oder Änderungen entstehen.

Kanban eignet sich für dieses Projekt besser, weil die Aufgaben flexibel bearbeitet werden können. Es gibt keine festen Sprint-Zeiträume und keine Verpflichtung, bestimmte Aufgaben innerhalb eines vorher definierten Zeitfensters abzuschließen. Stattdessen wird der aktuelle Arbeitsstand über das Kanban Board sichtbar gemacht.

4.3 Kanban Board und Tasks

Für die praktische Umsetzung wurde Jira als Kanban-Tool verwendet. Das Board besteht aus mehreren Spalten, die den aktuellen Zustand der Aufgaben zeigen.

Die verwendeten Spalten sind:

Spalte	Bedeutung
Backlog	Aufgaben, die geplant sind, aber noch nicht begonnen wurden
To Do	Aufgaben, die als Nächstes bearbeitet werden sollen
In Progress	Aufgaben, an denen aktuell gearbeitet wird
Review	Aufgaben, die überprüft oder getestet werden
Done	Abgeschlossene Aufgaben

Tabelle 5: Spalten des Kanban Boards

Die Anforderungen aus dem Requirements Engineering wurden in konkrete Tasks überführt. Dadurch wurde aus einer fachlichen Anforderung eine umsetzbare Entwicklungsaufgabe.

Task	Beschreibung
TASK-000 – Technische Vorbereitung / Projektvorbereitung	Vorbereitung der Entwicklungsumgebung, Repository und Projektgrundlage
TASK-001 – Projektstruktur und Module für ReconForge planen	Planung der modularen Projektstruktur
TASK-002 – Logging integrieren	Integration einer Logging-Lösung in das Projekt
TASK-003 – Domain Scan implementieren	Umsetzung der Domain-Scan-Funktion
TASK-004 – Subdomain Discovery implementieren	Umsetzung der Subdomain-Ermittlung
TASK-005 – IP Resolution implementieren	Auflösung von Domains und Subdomains in IP-Adressen

Task	Beschreibung
TASK-006 – Port Scan implementieren	Prüfung ausgewählter Ports einer IP-Adresse
TASK-007 – Export-System implementieren	Umsetzung eines generischen Export-Systems für JSON, CSV, XML, HTML und YAML
TASK-008 – CLI Interface implementieren	Umsetzung der Bedienung über die Kommandozeile
TASK-009 – Unit Testing Support einrichten	Einrichtung der Grundlage für Unit Tests

Tabelle 6: Kanban Tasks für ReconForge

4.4 Reflexion

Die Entscheidung für Kanban war für dieses Projekt sinnvoll, weil ReconForge flexibel und schrittweise umgesetzt wird. Besonders durch die Nutzung von Codex und Vibe Coding ist nicht immer im Voraus genau abschätzbar, wie lange einzelne Aufgaben dauern oder welche Anpassungen während der Umsetzung notwendig werden.

Kanban erlaubt es, Aufgaben nach ihrem aktuellen Bearbeitungsstand zu organisieren, ohne an feste Sprint-Zeiträume gebunden zu sein. Dadurch passt das Vorgehensmodell besser zu einem kleinen, modularen Projekt wie ReconForge.

Außerdem macht das Kanban Board den Fortschritt sichtbar. Dadurch kann leicht nachvollzogen werden, welche Aufgaben noch offen sind, welche gerade bearbeitet werden und welche bereits abgeschlossen wurden.

5. AI / Vibe Coding

5.1 Ziel des Vibe Coding

Im Rahmen des Projekts ReconForge wurde Vibe Coding als unterstützende Arbeitsweise eingesetzt. Dabei wurde die Implementierung nicht vollständig manuell und auch nicht vollständig automatisch durchgeführt. Stattdessen wurden einzelne Entwicklungsschritte mithilfe von KI vorbereitet, geprüft und anschließend in den Projektkontext eingeordnet.

Ziel war es, die Umsetzung schrittweise zu unterstützen und gleichzeitig die Kontrolle über Architektur, Anforderungen und Codequalität zu behalten. Die KI

wurde dabei nicht als Ersatz für die eigene Arbeit verwendet, sondern als Werkzeug zur Unterstützung bei der Planung, Implementierung, Strukturierung und Verbesserung einzelner Projektbestandteile.

Besonders wichtig war, dass nicht das gesamte Projekt auf einmal generiert wurde. Stattdessen wurden die Anforderungen aus dem Requirements Engineering und die Tasks aus dem Kanban Board in einzelne, klar abgegrenzte Prompts überführt. Dadurch blieb der Entwicklungsprozess nachvollziehbar und kontrollierbar.

5.2 Vergleich der verwendeten AI-Werkzeuge

Für die AI-gestützte Entwicklung wurden mehrere Werkzeuge betrachtet. Dabei wurde zwischen Werkzeugen für Dokumentation, Codegenerierung, UI-Prototyping und allgemeine Unterstützung unterschieden.

Werkzeug	Mögliche Verwendung	Bewertung für ReconForge
ChatGPT	Unterstützung bei Dokumentation, Formulierungen, Strukturierung und sprachlicher Korrektur	Sehr gut geeignet für die Projektdokumentation und Erklärung von Konzepten
Codex	Unterstützung bei der schrittweisen Implementierung, Codegenerierung, Tests und Refactoring	Sehr gut geeignet für Vibe Coding und die Umsetzung einzelner Programmieraufgaben
GitHub Copilot	Codevorschläge direkt im Editor	Hilfreich für kleine Codeergänzungen, aber weniger geeignet für längere strukturierte Arbeitsaufträge
Lovable / Bolt / Replit	Schnelles Erstellen von Prototypen oder Webanwendungen	Für ReconForge weniger passend, da es sich nicht um eine klassische Webanwendung handelt
Google Stitch	Unterstützung bei UI-Entwürfen	Für ReconForge nur eingeschränkt relevant, da das Projekt ein Command-Line-Tool ist

Tabelle 7: Vergleich der verwendeten AI-Werkzeuge

Für die Dokumentation wurde hauptsächlich ChatGPT verwendet. Dabei ging es vor allem um die Formulierung von Texten, die sprachliche Korrektur und die Strukturierung der Projektdokumentation.

Für die Implementierung wurde Codex ausgewählt, weil ReconForge als C#/NET-Projekt schrittweise umgesetzt werden sollte. Codex eignet sich besonders gut für längere, strukturierte Programmieraufgaben, bei denen bestehender Code erweitert und nicht einfach ein komplett neues Projekt erzeugt werden soll.

5.3 Entscheidung für Codex

Codex wurde für die Umsetzung gewählt, weil das Projekt aus mehreren klar getrennten Modulen besteht. Dazu gehören unter anderem CLI, Domain Scan, Subdomain Discovery, IP Resolution, Port Scan, Export, Logging und Tests.

Diese modulare Struktur passt gut zu einer promptbasierten Vorgehensweise. Jede Funktion konnte einzeln beschrieben und anschließend umgesetzt werden. Dadurch war es möglich, die Entwicklung kontrolliert und nachvollziehbar aufzubauen.

Ein weiterer Grund für Codex war, dass bei ReconForge keine visuelle Benutzeroberfläche im Vordergrund steht. Das Projekt ist ein Command-Line-Tool. Deshalb waren Werkzeuge für Web- oder UI-Prototyping weniger geeignet. Wichtiger waren saubere Architektur, verständliche Klassen, getrennte Verantwortlichkeiten, Tests und Erweiterbarkeit.

Codex wurde also vor allem für die codebezogene Umsetzung verwendet, während ChatGPT für die Dokumentation, Erklärungen und sprachliche Ausarbeitung genutzt wurde.

5.4 Vorgehensweise bei der Prompt-Erstellung

Die Prompts wurden bewusst sehr detailliert formuliert. Jeder Prompt beschreibt ein konkretes Ziel, den aktuellen Projektstand, die gewünschte Erweiterung, technische Vorgaben, Einschränkungen und erwartete Ergebnisse.

Dadurch sollte verhindert werden, dass die KI zu viel auf einmal verändert oder Funktionen vermischt. Zum Beispiel wurde beim Domain Scan ausdrücklich festgelegt, dass noch keine Subdomain Discovery, IP Resolution oder Port Scanning umgesetzt werden soll. Ebenso wurde beim Port Scan darauf geachtet, dass nur ein einfacher und kontrollierter Scan für Lernzwecke implementiert wird.

Die Prompts wurden so aufgebaut, dass Codex immer auf dem vorherigen Stand weiterarbeiten konnte. Dadurch entstand ein schrittweiser

Entwicklungsprozess, der gut zum Kanban-Ansatz passt.

5.5 Beschreibung der verwendeten Prompts

Die vollständigen Prompts der AI-gestützten Umsetzung befinden sich im GitHub-Repository im Ordner `docs/prompts`. Dadurch bleibt die Projektdokumentation übersichtlich, während die verwendeten Prompts weiterhin nachvollziehbar dokumentiert sind.

<https://github.com/mohamedderki/ReconForge/tree/main/docs/prompts>

Prompt 1 – Projektstruktur und Grundarchitektur

Der erste Prompt diente dazu, die Grundstruktur von ReconForge zu erstellen. Ziel war es nicht, sofort alle Funktionen vollständig umzusetzen, sondern zuerst eine saubere und modulare Projektbasis zu schaffen.

Dabei wurden die wichtigsten Bereiche des Projekts vorbereitet, zum Beispiel CLI, Core, Scanning, Export, Infrastructure und Tests. Außerdem wurde festgelegt, dass die Geschäftslogik von der CLI getrennt bleiben soll. Diese Trennung ist wichtig, damit das Projekt später leichter erweitert, getestet und gewartet werden kann.

Der Prompt bildete damit die Grundlage für alle weiteren Implementierungsschritte.

Prompt 2 – CLI-Grundlage und Logging

Der zweite Prompt baute auf der vorhandenen Projektstruktur auf. Ziel war es, eine erste lauffähige CLI-Grundlage zu schaffen. Dabei sollte ReconForge später wie ein echtes Command-Line-Tool verwendet werden können, zum Beispiel mit einer Option wie `--scan`.

Zusätzlich wurde Logging integriert. Dadurch können wichtige Abläufe, Fehler und Ergebnisse nachvollziehbar ausgegeben werden. Logging ist besonders hilfreich, weil bei einem Reconnaissance-Tool mehrere Verarbeitungsschritte nacheinander ablaufen.

Wichtig war in diesem Schritt, dass die CLI nur die Bedienung koordiniert und die eigentliche Logik weiterhin in eigenen Services bleibt.

Prompt 3 – Domain Scan

Mit dem dritten Prompt wurde die erste fachliche Funktion umgesetzt: der Domain Scan. Dabei ging es noch nicht um echte Netzwerkoperationen,

sondern um die Verarbeitung einer Domain als Eingabe.

Die Domain sollte validiert, normalisiert und als Grundlage für spätere Schritte vorbereitet werden. Dadurch wurde eine klare Basis für Subdomain Discovery, IP Resolution und Port Scan geschaffen.

Der Prompt war bewusst begrenzt, damit die Verantwortlichkeiten sauber getrennt bleiben. Domain Scan sollte nur für Domain-Verarbeitung zuständig sein und keine anderen Funktionen übernehmen.

Prompt 4 – Subdomain Discovery

Der vierte Prompt erweiterte den bestehenden Scan-Ablauf um Subdomain Discovery. Dabei wurden für eine gültige Domain mögliche Subdomains erzeugt oder gesammelt.

Die Umsetzung wurde bewusst einfach und kontrolliert gehalten. Es wurden keine großen Wordlists, kein aggressives Brute-Forcing und keine externen APIs verwendet. Stattdessen ging es darum, das Modul sauber in den bestehenden Workflow einzubauen.

Dieser Schritt zeigt gut den modularen Aufbau des Projekts: Die Domain wird zuerst verarbeitet, danach werden Subdomains ermittelt, ohne bereits IP-Adressen oder Ports zu behandeln.

Prompt 5 – IP Resolution

Mit dem fünften Prompt wurde die IP Resolution ergänzt. Nach der Domain-Verarbeitung und der Subdomain Discovery sollten die Domain und die gefundenen Subdomains in IP-Adressen aufgelöst werden.

Dabei wurde darauf geachtet, dass DNS-Fehler sauber behandelt werden. Wenn ein Host nicht aufgelöst werden kann, soll das Programm nicht abbrechen, sondern den Fehler erfassen und mit den übrigen Hosts fortfahren.

Auch hier blieb die Verantwortung klar getrennt: IP Resolution löst nur Hostnamen in IP-Adressen auf und führt noch keinen Port Scan durch.

Prompt 6 – Safe Port Scan

Der sechste Prompt ergänzte einen einfachen und kontrollierten Port Scan. Dabei sollten nur ausgewählte Ports geprüft werden. Aggressive Scan-Techniken, vollständige Portbereiche oder sicherheitskritische Angriffsfunktionen wurden bewusst ausgeschlossen.

Der Port Scan wurde als Lern- und Demonstrationsfunktion geplant. Ziel war es, zu prüfen, ob bestimmte Ports erreichbar sind, ohne das Tool zu einem aggressiven Sicherheitsscanner zu machen.

Dieser Prompt war besonders wichtig, weil ReconForge im Bereich IT-Sicherheit liegt. Deshalb wurde klar festgelegt, dass das Tool nur für autorisierte Tests und Lernzwecke gedacht ist.

Prompt 7 – Export-System mit Factory Pattern

Der siebte Prompt behandelte den Export der Scan-Ergebnisse. Ursprünglich war nur JSON Export als Requirement vorgesehen. Die Architektur wurde jedoch so geplant, dass weitere Exportformate einfach ergänzt werden können.

Dafür wurde eine generische Export-Schnittstelle und das Factory Pattern verwendet. Die Factory wählt anhand des gewünschten Formats den passenden Exporter aus. Dadurch muss der Scan-Ablauf nicht wissen, ob die Ergebnisse als JSON, CSV, XML, HTML oder YAML exportiert werden.

Diese Lösung unterstützt besonders die nicht-funktionale Anforderung Erweiterbarkeit. Neue Exportformate können später ergänzt werden, ohne die bestehende Scan-Logik stark zu verändern.

Prompt 8 – Unit Testing Support

Der achte Prompt konzentrierte sich auf Testbarkeit und Qualitätssicherung. Ziel war es, die wichtigsten Module durch Unit Tests überprüfbar zu machen.

Dabei sollten insbesondere Domain-Validierung, Subdomain Discovery, IP Resolution, Port Scan und Export getestet werden. Für externe Abhängigkeiten wie DNS-Abfragen oder Netzwerkverbindungen sollten Abstraktionen verwendet werden, damit Tests stabil bleiben und nicht vom echten Netzwerk abhängig sind.

Dieser Schritt unterstützt die nicht-funktionale Anforderung Testbarkeit und verbessert die Wartbarkeit des Projekts.

Prompt 9 – CLI-Banner und visuelles Design

Der neunte Prompt ergänzte einen professionellen CLI-Banner für ReconForge. Ziel war es, dem Tool eine erkennbare visuelle Identität zu geben, ohne die Lesbarkeit oder Testbarkeit zu verschlechtern.

Dabei wurde darauf geachtet, dass der Banner nur zur CLI-Schicht gehört und nicht mit der Geschäftslogik vermischt wird. Außerdem sollte der Banner über

eine Umgebungsvariable deaktiviert werden können, damit automatisierte Tests und CI-Ausgaben übersichtlich bleiben.

Auch Farben und Design wurden bewusst geplant. Der Banner sollte technisch, sauber und passend zu einem Reconnaissance-Tool wirken.

Prompt 10 – Terminalausgabe als Standard und optionaler Export

In einem weiteren Schritt wurde das Verhalten der CLI angepasst.

Standardmäßig sollen die Ergebnisse eines Scans direkt im Terminal angezeigt werden. Ein Export in eine Datei soll nicht automatisch erfolgen, sondern nur dann, wenn der Benutzer dies ausdrücklich über eine Export-Option auswählt.

Dadurch verhält sich ReconForge wie ein typisches Command-Line-Tool: Die direkte Ausgabe im Terminal steht im Vordergrund, während Datei-Exporte zusätzliche optionale Funktionen sind. Für die strukturierte Darstellung der Ergebnisse im Terminal wird Spectre.Console verwendet. Die Export-Komponente bleibt weiterhin vorhanden und kann bei Bedarf für JSON, CSV, XML, HTML oder YAML genutzt werden.

5.6 Implementierung mit Codex

Die Implementierung von ReconForge erfolgte schrittweise mithilfe von Codex. Dabei wurde nicht das gesamte Projekt auf einmal erzeugt, sondern jede Funktion wurde einzeln geplant, beschrieben und umgesetzt. Grundlage dafür waren die zuvor definierten Anforderungen sowie die daraus abgeleiteten Kanban Tasks.

Zu Beginn wurde zuerst die Projektstruktur vorbereitet. Dabei wurden die wichtigsten Bereiche des Projekts festgelegt, zum Beispiel CLI, Core, Scanning, Export, Infrastructure und Tests. Diese Struktur bildete die Grundlage für die spätere modulare Umsetzung.

Anschließend wurde die CLI-Grundlage umgesetzt. ReconForge sollte als Command-Line-Tool bedienbar sein und über Optionen wie `--scan`, `--verbose`, `-export`, `--format` und `--output` gesteuert werden können. Gleichzeitig wurde Logging integriert, damit wichtige Abläufe, Fehler und Ergebnisse nachvollziehbar ausgegeben werden können.

Danach wurden die fachlichen Funktionen Schritt für Schritt ergänzt. Zuerst wurde der Domain Scan umgesetzt. Dabei ging es um die Verarbeitung, Validierung und Normalisierung einer eingegebenen Domain. Im nächsten

Schritt wurde die Subdomain Discovery ergänzt. Danach folgten IP Resolution und ein einfacher, kontrollierter Port Scan.

Nach den Scan-Funktionen wurde das Export-Modul umgesetzt. Dabei wurde nicht nur ein einzelner JSON-Export erstellt, sondern eine generische Export-Struktur mit Factory Pattern geplant. Dadurch können verschiedene Exportformate wie JSON, CSV, XML, HTML und YAML unterstützt werden, ohne die Scan-Logik direkt an ein bestimmtes Exportformat zu binden.

Im weiteren Verlauf wurden Unit Tests ergänzt, um wichtige Funktionen des Projekts überprüfbar zu machen. Besonders die Domain-Validierung, die Subdomain Discovery, die IP Resolution, der Port Scan und das Export-System wurden dabei berücksichtigt.

Zusätzlich wurde ein CLI-Banner ergänzt, um dem Tool eine erkennbare visuelle Identität zu geben. Abschließend wurde das Verhalten der CLI so angepasst, dass die Ergebnisse standardmäßig direkt im Terminal angezeigt werden. Ein Export in eine Datei erfolgt nur dann, wenn der Benutzer dies ausdrücklich über die Option `--export` auswählt.

Die Arbeit mit Codex wurde kontrolliert durchgeführt. Die erzeugten Codevorschläge wurden nicht ungeprüft übernommen, sondern geprüft, angepasst und in die bestehende Projektstruktur eingeordnet.

5.7 Ergebnisse der Implementierung

Durch die schrittweise Umsetzung mit Codex entstand eine lauffähige Grundlage für das Projekt ReconForge. Die Implementierung wurde in mehrere kleinere Aufgaben aufgeteilt, sodass die einzelnen Funktionen kontrolliert entwickelt und nachvollziehbar dokumentiert werden konnten.

Als Ergebnis wurde zunächst eine modulare Projektstruktur erstellt. Die wichtigsten Bereiche wie CLI, Core, Scanning, Export, Infrastructure und Tests wurden voneinander getrennt. Dadurch ist das Projekt übersichtlich aufgebaut und spätere Erweiterungen können einfacher ergänzt werden.

Die CLI-Grundlage wurde so umgesetzt, dass ReconForge über Kommandozeilenoptionen bedient werden kann. Der Benutzer kann einen Scan über eine Option wie `--scan` starten und weitere Einstellungen wie `--verbose`, `--export`, `--format` oder `--output` angeben.

Die zentralen Scan-Funktionen wurden ebenfalls schrittweise ergänzt. Dazu gehören die Verarbeitung und Validierung einer Domain, die Subdomain

Discovery, die IP Resolution sowie ein einfacher und kontrollierter Port Scan. Die einzelnen Funktionen wurden jeweils in eigenen Services beziehungsweise Modulen umgesetzt, damit die Verantwortlichkeiten klar getrennt bleiben.

Ein weiteres Ergebnis ist das Export-Modul. Die Scan-Ergebnisse können in verschiedenen Formaten exportiert werden, zum Beispiel JSON, CSV, XML, HTML und YAML. Dafür wurde eine generische Export-Struktur mit Factory Pattern verwendet. Dadurch kann das Projekt später leichter um weitere Exportformate erweitert werden.

Zusätzlich wurden Unit Tests ergänzt, um wichtige Teile des Projekts überprüfbar zu machen. Besonders die Domain-Validierung, die Subdomain Discovery, die IP Resolution, der Port Scan und das Export-System wurden dabei berücksichtigt.

Als sichtbares Ergebnis wurde außerdem ein CLI-Banner ergänzt. Dieser dient als visuelle Wiedererkennung des Tools und enthält einen Hinweis darauf, dass ReconForge nur für Lernzwecke und autorisierte Tests gedacht ist.

Besonders wichtig ist, dass die Scan-Ergebnisse standardmäßig direkt im Terminal angezeigt werden. Ein Datei-Export erfolgt nur optional, wenn der Benutzer dies ausdrücklich auswählt. Dadurch verhält sich ReconForge wie ein typisches Command-Line-Tool: Terminalausgabe im Standardfall und Export nur bei Bedarf.

Insgesamt entstand durch die AI-gestützte Umsetzung eine strukturierte und erweiterbare Grundlage für ReconForge. Die wichtigsten Anforderungen des Projekts wurden in einzelne Module überführt und Schritt für Schritt umgesetzt.

Die zugehörigen Screenshots der AI-gestützten Umsetzung befinden sich im GitHub-Repository im Ordner `Screenshots/Codex`.

<https://github.com/mohamedderki/ReconForge/tree/main/Screenshots/Codex>

5.8 Reflexion zum Vibe Coding

Der Einsatz von Codex war besonders hilfreich, weil die Entwicklung von ReconForge in kleine, klar abgegrenzte Schritte zerlegt wurde. Dadurch konnten einzelne Funktionen nacheinander umgesetzt werden, ohne das gesamte Projekt unkontrolliert auf einmal generieren zu lassen.

Gleichzeitig zeigte sich, dass Vibe Coding nur dann sinnvoll ist, wenn die Prompts präzise formuliert werden. Allgemeine oder zu kurze Prompts können

dazu führen, dass die KI unpassende Architekturentscheidungen trifft oder zu viele Funktionen miteinander vermischt.

Durch die detaillierten Prompts blieb die Kontrolle über das Projekt erhalten. Jede erzeugte Lösung musste geprüft, angepasst und in den bestehenden Code integriert werden. Besonders wichtig war dabei die Trennung der Verantwortlichkeiten zwischen CLI, Scanning, Export, Logging und Tests.

Insgesamt war Vibe Coding mit Codex für ReconForge gut geeignet, weil das Projekt modular aufgebaut ist. Die KI konnte einzelne Umsetzungsschritte vorbereiten, während die fachliche Entscheidung, Bewertung und Verantwortung beim Entwickler blieb.

6. Fazit

6.1 Zusammenfassung

Im Rahmen dieses Projekts wurde ReconForge als modulares Command-Line-Tool geplant, dokumentiert und schrittweise umgesetzt. Das Projekt diente dazu, verschiedene Methoden der Softwaretechnik praktisch anzuwenden und an einem konkreten Beispiel nachvollziehbar zu machen.

Zu Beginn wurden Anforderungen gesammelt, geordnet und in funktionale sowie nicht-funktionale Anforderungen eingeteilt. Anschließend wurden die Anforderungen in Notion dokumentiert und in Jira in konkrete Aufgaben für ein Kanban Board überführt.

Mithilfe von UML-Diagrammen wurde das Projekt aus verschiedenen Perspektiven betrachtet. Das Use-Case-Diagramm zeigt die wichtigsten Benutzeraktionen, das Class-Diagramm beschreibt zentrale Datenstrukturen, das Activity-Diagramm zeigt den Ablauf des Scan-Prozesses, das Sequence-Diagramm stellt die Kommunikation zwischen den Komponenten dar und das Component-Diagramm beschreibt die modulare Architektur.

Für die Projektplanung wurde Kanban gewählt, da ReconForge schrittweise und flexibel umgesetzt wurde. Besonders im Zusammenhang mit Vibe Coding und der Nutzung von Codex war Kanban besser geeignet als ein Scrum-Ansatz mit festen Sprint-Zeiträumen.

Im AI-Teil wurde gezeigt, wie Codex unterstützend für die Umsetzung einzelner Funktionen genutzt wurde. Dabei wurden detaillierte Prompts erstellt, um die Entwicklung kontrolliert und nachvollziehbar zu halten. Die KI wurde nicht als

Ersatz für eigene Arbeit eingesetzt, sondern als Werkzeug zur Unterstützung bei Planung, Implementierung und Strukturierung.

6.2 Erfahrungen

Während der Bearbeitung wurde deutlich, dass eine gute Struktur für ein Softwareprojekt sehr wichtig ist. Besonders die Trennung zwischen CLI, Scanning, Export, Logging und Tests hat geholfen, das Projekt übersichtlich und erweiterbar zu halten.

Auch die Arbeit mit Anforderungen war hilfreich, weil dadurch klarer wurde, welche Funktionen das System wirklich bereitstellen soll. Die Unterscheidung zwischen funktionalen und nicht-funktionalen Anforderungen hat dabei geholfen, nicht nur die Funktionen, sondern auch Qualitätsmerkmale wie Wartbarkeit, Erweiterbarkeit und Testbarkeit zu berücksichtigen.

Die Nutzung von Kanban war für dieses Projekt sinnvoll, weil Aufgaben flexibel nacheinander bearbeitet werden konnten. Dadurch passte das Vorgehensmodell gut zur schrittweisen Umsetzung und zur Arbeit mit Codex.

Eine wichtige Erfahrung war außerdem, dass KI-Werkzeuge nur dann gute Ergebnisse liefern, wenn die Prompts präzise formuliert sind. Zu allgemeine Prompts können zu unpassenden oder zu umfangreichen Ergebnissen führen. Detaillierte Prompts helfen dagegen, die Kontrolle über Architektur, Funktionsumfang und Codequalität zu behalten.

6.3 Ausblick

ReconForge kann in Zukunft weiter ausgebaut werden. Denkbar sind zusätzliche Scan-Module, weitere Exportformate, eine verbesserte Konfiguration oder eine ausführlichere Scan-Historie.

Auch die Testabdeckung kann weiter verbessert werden. Besonders sinnvoll wäre es, zusätzliche Tests für den vollständigen Workflow und für verschiedene Fehlerfälle zu ergänzen.

Außerdem könnte das Projekt später um eine bessere Paketierung erweitert werden, sodass ReconForge direkt als ausführbares CLI-Tool über den Programmnamen gestartet werden kann. Dadurch wäre die Nutzung noch näher an einem echten Command-Line-Tool.

Insgesamt bietet ReconForge eine gute Grundlage, um Methoden der Softwaretechnik praktisch anzuwenden und das Projekt schrittweise

weiterzuentwickeln.